



Scope. This document describes the architecture of the Dual Job Dating mobile app: its component layering, technology choices, and the data model of the backend persistence layer. Operational topics (CI/CD, distribution, secrets) are covered in the README and the Technical Handover.

OVERVIEW

Dual Job Dating is a cross-platform mobile application built with **Kotlin Multiplatform** and **Compose Multiplatform**. All application code lives in `commonMain`; platform-specific code is limited to the Ktor HTTP engine (OkHttp on Android, Darwin on iOS) and one native dialog on Android. The app talks to two external systems: a custom Go backend exposing the domain API and Supabase for authentication, file storage, and the underlying Postgres database.

TECH STACK

Layer	Technology	Version
Framework	Kotlin Multiplatform	2.3.0
UI	Compose Multiplatform	1.10.0
Architecture	MVVM + Feature Modules	-
Dependency Injection	Koin	4.1.0
Networking	Ktor Client (OkHttp / Darwin)	3.1.3
Navigation	Jetpack Navigation Compose (type-safe routes)	2.9.1
Backend SDK	Supabase (Auth, PostgREST, Storage, Realtime)	3.1.4
Local storage	Multiplatform Settings	1.1.1
Image loading	Coil	3.2.0
Serialization	Kotlinx Serialization	-
Build-time config	BuildKonfig	0.15.2
Platforms	Android (min SDK 24), iOS 16+	-

COMPONENT DIAGRAM

The diagram below shows the runtime composition of the app. Each feature is a vertical slice from UI to data layer; cross-cutting concerns (DI, navigation, network, session, theme) live in `core`. The platform shims at the bottom provide the concrete Ktor engine and any platform dialogs the common code requires.

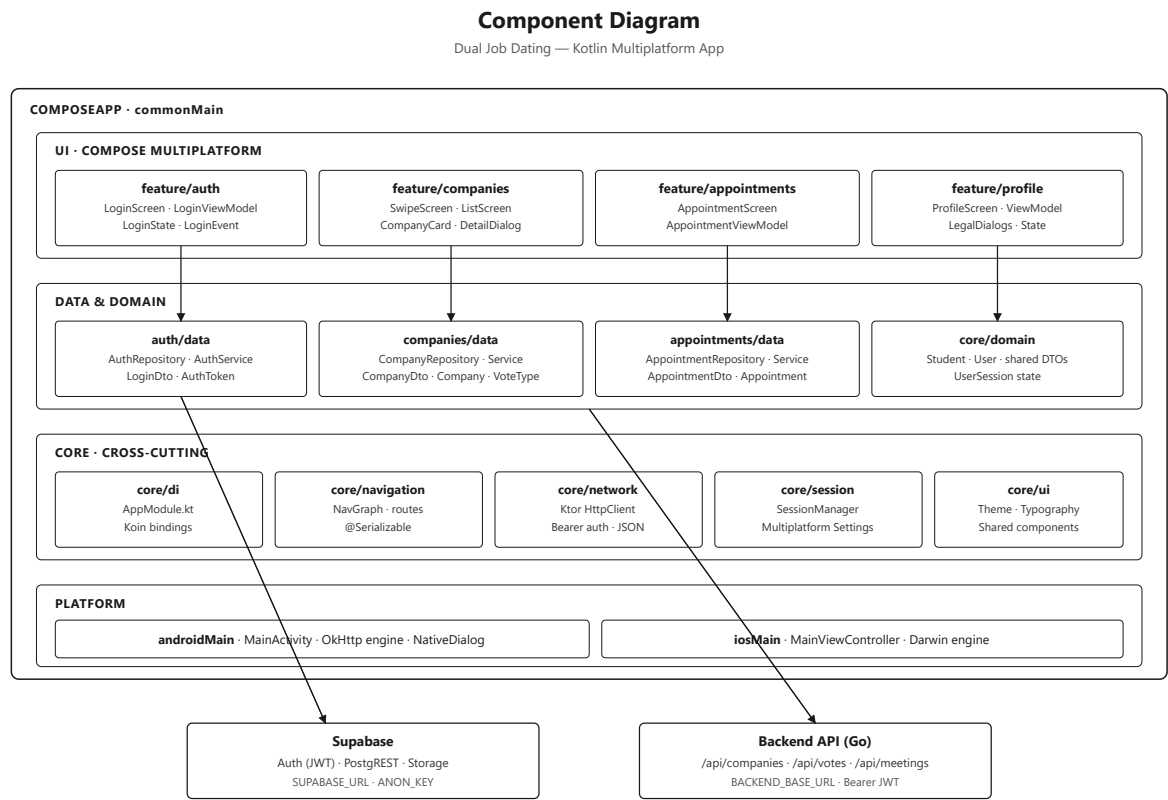


Figure 1 - Component layering of the Compose Multiplatform app.

Layer responsibilities

Layer	Responsibility
feature/*/ui	Composable screens, ViewModels, immutable <code>State</code> data classes, sealed <code>Event</code> classes. No business logic in screens.
feature/*/data	<code>Repository</code> interfaces (the only thing a ViewModel depends on) and <code>Service</code> implementations that do raw API calls. DTOs are mapped to domain models exclusively in <code>RepositoryImpl</code> .
core/di	Koin <code>AppModule.kt</code> binds all interfaces to implementations. Swapping a service requires no changes elsewhere.
core/navigation	Single <code>NavGraph</code> with <code>@Serializable</code> route objects. Navigation side effects are emitted via <code>MutableSharedFlow</code> from ViewModels.
core/network	Ktor <code>HttpClient</code> setup, base URL injection, bearer-token authentication, JSON content negotiation, error mapping.
core/session	<code>SessionManager</code> persists the JWT and refresh token through Multiplatform Settings for auto-login on subsequent launches.
core/ui	<code>DualJobDatingTheme</code> , typography, color palette, shared composables (bottom navigation, app background, dialogs).
androidMain / iosMain	Activity / ViewController entry points, the platform-specific Ktor engine factory, and one native confirmation dialog on Android.

Data flow per user interaction

A user action is dispatched as a sealed `Event` to the screen's `onEvent()` handler, which calls the ViewModel. The ViewModel invokes its `Repository`, which delegates to a `Service` that performs the actual Supabase SDK call or Ktor HTTP request. The result is mapped from DTO to domain model inside the repository and used to update the ViewModel's `MutableStateFlow`. The screen renders the new state via `collectAsState()`. One-time side effects (navigation, snackbar) are emitted on a separate `MutableSharedFlow` consumed inside a `LaunchedEffect`.



DATA MODEL

Persistence lives in the Supabase-hosted Postgres database. The schema below is reconstructed from the OpenAPI specification (`docs/tech/backend_api.yaml`) and the DTO classes in the Kotlin codebase. Authentication identities are managed by Supabase Auth; `users.id` is the same UUID Supabase issues for the authenticated principal.

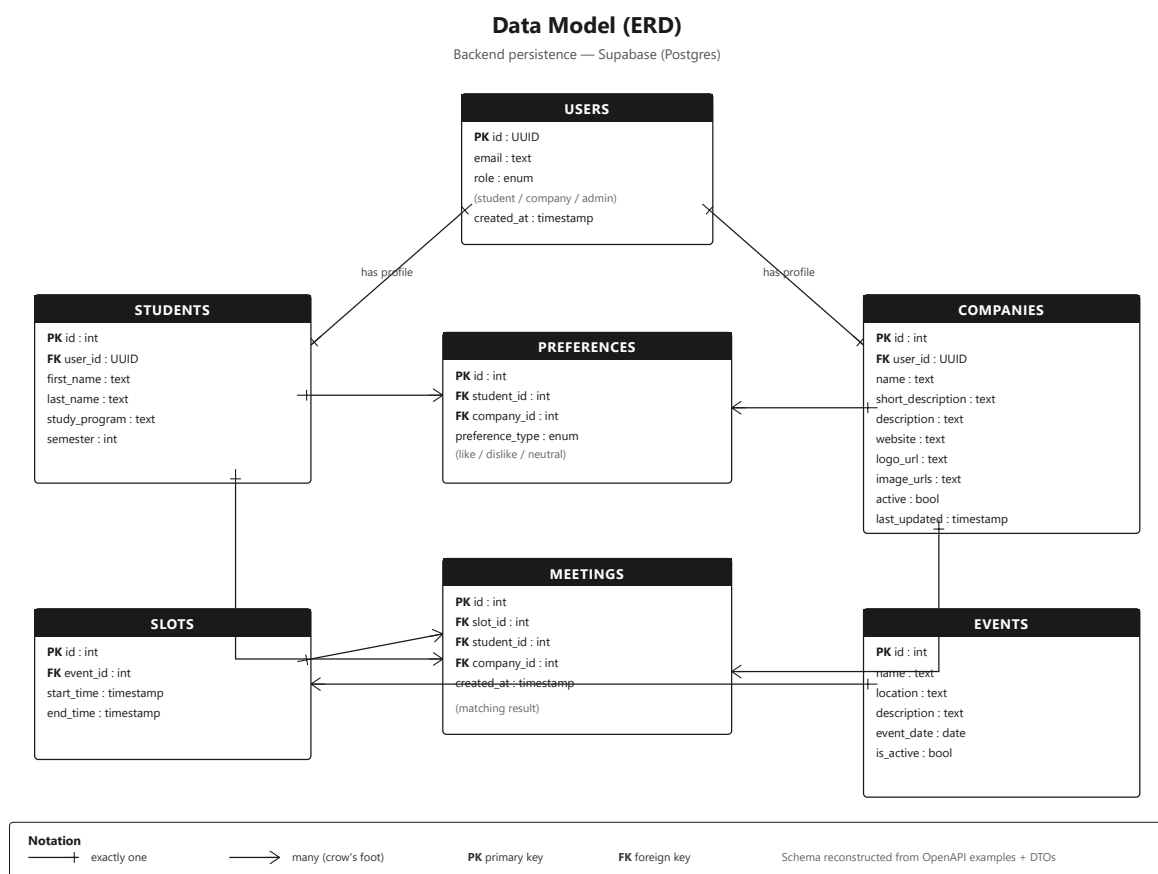


Figure 2 - Entity Relationship Diagram (crow's foot notation).

Entities

Entity	Purpose
users	Authentication identity (Supabase Auth). Distinguishes role (student / company / admin). One <code>users</code> row corresponds to exactly one <code>students</code> or <code>companies</code> profile.
students	Profile of a participating student. Holds name, study program, and semester. The <code>user_id</code> links to the authenticated identity.
companies	Profile of a participating company. Holds public description, website, logo, and a semicolon-separated list of additional image URLs. <code>active</code> controls whether the company appears in the swipe stack.
preferences	A single vote of one student on one company. <code>preference_type</code> is one of <i>like</i> , <i>dislike</i> , or <i>neutral</i> . Produced by the swipe screen; consumed by the matching algorithm.
events	A Job Dating event (date, location, activation flag). Owns the time slots for that day.
slots	A single time slot within an event (start / end timestamp). Many slots per event.
meetings	The output of the matching process. Assigns one student and one company to one slot. Read-only from the app's perspective and displayed on the Appointments screen.

Cardinalities at a glance

- `users` 1 — 1 `students` (role = student)
- `users` 1 — 1 `companies` (role = company)
- `students` 1 — M `preferences` M — 1 `companies`
- `events` 1 — M `slots`
- `slots` 1 — M `meetings`
- `students` 1 — M `meetings` M — 1 `companies`

API surface

The DTOs the app consumes correspond directly to the entities above. The most important endpoints are:

Endpoint	Used by
POST /auth/v1/token (Supabase)	AuthService - login, refresh
GET /api/companies	CompanyService - swipe stack & list view
GET /api/students/{id}/preferences · POST /api/preferences	CompanyRepository - vote submission
GET /api/students/{id}/meetings	AppointmentService - schedule on Appointments screen
PATCH /api/users/{id}	ProfileViewModel - profile updates